

PROJETO AVANÇADO - CAPACITA BRASIL / IREDE

AWS E-commerce com Terraform

Material Completo de Estudo

Provisionamento de Serviços Computacionais - Computação em Nuvem

ARQUITETURA IMPLEMENTADA

Usuário -> EC2 (APIs Node.js) -> SQS -> Lambda -> CloudWatch Logs

Infraestrutura como Código com Terraform

REPOSITÓRIO DO PROJETO

github.com/charbeldecastro/aws-ecommerce-terraform

Documentação de estudo baseada na implementação prática do Projeto Avançado

Versão para portfólio e consulta técnica - Julho de 2026

Como usar esta apostila

Este material reconstrói o projeto executado na conversa e o transforma em uma apostila de estudo. Ele separa fatos confirmados do projeto, explicações complementares e boas práticas profissionais.

FIDELIDADE AO HISTÓRICO

Realizado no projeto: confirmado pelas mensagens, comandos ou capturas de tela. Explicação complementar: conhecimento necessário para entender o que foi feito. Boa prática recomendada: orientação profissional que pode não ter sido executada durante a atividade.

DOCKER

Não há evidência de Docker ou Dockerfile na execução confirmada deste projeto. As APIs foram executadas diretamente com Node.js na EC2. Docker é explicado apenas como conhecimento complementar e para eliminar a confusão.

Arquitetura final confirmada

Usuário -> Internet -> EC2 (APIs Node.js) -> Amazon SQS -> AWS Lambda -> CloudWatch Logs

Sumário

1. Visão geral e requisitos do Projeto Avançado
2. Arquitetura e fluxo completo
3. Fundamentos das tecnologias usadas
4. Organização do repositório e arquivos
5. Terraform e Infraestrutura como Código
6. Rede AWS: VPC, subnet, rotas e Security Group
7. EC2, Linux, SSH e execução das APIs
8. API de Produtos
9. API de Pedidos e Amazon SQS
10. IAM: permissões entre EC2, SQS, Lambda e CloudWatch
11. AWS Lambda e CloudWatch Logs
12. Testes ponta a ponta e evidências
13. Git, GitHub, README e entrega
14. Projeto do zero ao deploy
15. Erros reais e troubleshooting
16. Segurança e boas práticas
17. Docker e Dockerfile: o que não usamos
18. Custos AWS e encerramento seguro
19. Glossário
20. Folha de consulta rápida de comandos
21. Perguntas de revisão e gabarito comentado
22. Simulado final - 30 questões
23. Gabarito comentado do simulado
24. Desafio final: refaça o projeto sozinho

1. Visão geral e requisitos do Projeto Avançado

O objetivo foi atuar como desenvolvedor de infraestrutura e construir, na AWS, uma solução de e-commerce baseada em duas APIs e comunicação assíncrona. A infraestrutura deveria ser criada com Terraform, demonstrando Infraestrutura como Código (IaC).

Entrega confirmada

- Repositório GitHub com código Terraform completo.
- Código das APIs.
- README.md explicando arquitetura, execução do Terraform, acesso à aplicação e teste do fluxo de pedidos.
- Evidências: EC2 com aplicação rodando, mensagem no SQS, log da Lambda no CloudWatch e saída do terraform apply.
- Encerramento com terraform destroy.

CONEXÃO COM AWS/CLOUD

O projeto combina IaaS (EC2 e rede), serviços gerenciados (SQS), serverless (Lambda) e observabilidade (CloudWatch), todos provisionados por IaC.

Perguntas de revisão

1. Qual era o papel do Terraform no projeto?
2. Quais eram as duas APIs exigidas?
3. Por que o SQS torna o fluxo assíncrono?
4. Quais evidências precisavam aparecer no README?

2. Arquitetura e fluxo completo

A arquitetura real não utilizou container. O usuário acessava a instância EC2 pelo IPv4 público. Na EC2 rodavam as APIs Node.js. A API de Pedidos enviava mensagens para uma fila SQS. A fila acionava uma função Lambda, e a função registrava o processamento no CloudWatch Logs.

Diagrama visual



Leitura do diagrama

- O Security Group controlava as portas de entrada da EC2.
- A API de Produtos respondia a requisições HTTP na porta 3000.
- A API de Pedidos respondia na porta 3001 e usava o AWS SDK para enviar ao SQS.
- IAM autorizava a comunicação sem depender de credenciais fixas no código.
- O event source mapping conectava SQS à Lambda.
- CloudWatch guardava os logs gerados pela execução da Lambda.

Perguntas de revisão

1. Em qual recurso as APIs foram executadas?
2. Qual componente desacoplava a API de Pedidos da Lambda?
3. Onde os logs do processamento ficaram visíveis?
4. Por que o Security Group faz parte da arquitetura mesmo não aparecendo no fluxo de negócio?

3. Fundamentos das tecnologias usadas

AWS

Plataforma de nuvem utilizada para hospedar rede, servidor, mensageria, função serverless, permissões e logs. Neste projeto, a AWS foi o ambiente onde toda a infraestrutura provisionada pelo Terraform existiu.

Terraform

Ferramenta de IaC. Os arquivos .tf descrevem o estado desejado. O fluxo típico é `init -> fmt -> validate -> plan -> apply`. O state permite ao Terraform relacionar configuração e recursos reais.

EC2

Máquina virtual na AWS. Foi o servidor das APIs Node.js. Tinha IP privado dentro da VPC e IPv4 público para acesso externo.

VPC

Rede virtual isolada. Agrupou subnet, roteamento e recursos de rede do projeto.

Subnet pública

Segmento da VPC associado a uma rota para Internet Gateway. Permitiu que a EC2 tivesse conectividade adequada para o laboratório.

Internet Gateway

Componente que conecta a VPC à internet quando as rotas e endereçamento permitem.

Route Table

Tabela que decide para onde o tráfego da subnet deve ser encaminhado. A rota 0.0.0.0/0 apontava para o Internet Gateway.

Security Group

Firewall stateful associado à EC2. No projeto foram abertas as portas 22, 80, 3000 e 3001. A abertura ampla para 0.0.0.0/0 foi funcional para laboratório, mas SSH deveria ser restrito em produção.

IAM

Sistema de identidade e autorização. Roles e policies deram à EC2 permissão para enviar ao SQS e à Lambda permissão para consumir SQS e escrever logs.

SQS

Fila gerenciada. Recebia pedidos e os mantinha desacoplados do processamento posterior.

Lambda

Função serverless acionada pelo SQS. Processava a mensagem sem servidor dedicado para essa etapa.

CloudWatch Logs

Serviço de logs. Registrou o evento recebido e o pedido processado pela Lambda.

Node.js + Express

Runtime e framework das APIs. As aplicações foram executadas diretamente na EC2.

AWS SDK for JavaScript

Biblioteca usada pela API de Pedidos para chamar o SQS programaticamente.

Git e GitHub

Git versionou o código; GitHub hospedou o repositório entregue ao professor.

CONEXÃO COM AWS/CLOUD

SQS e Lambda exemplificam desacoplamento e arquitetura orientada a eventos; IAM exemplifica responsabilidade compartilhada e controle de acesso; Terraform exemplifica automação e repetibilidade.

Perguntas de revisão

1. Qual diferença prática existe entre EC2 e Lambda neste projeto?
2. Por que IAM é preferível a colocar chaves AWS no código?
3. Qual papel a Route Table exerceu?
4. O que significa dizer que Security Group é stateful?

4. Organização do repositório e arquivos

```
aws-ecommerce-terraform/  
|-- lambda/  
|   |-- index.js  
|-- products-api/  
|   |-- index.js  
|   |-- package.json  
|   |-- package-lock.json  
|-- orders-api/  
|   |-- index.js  
|   |-- package.json  
|   |-- package-lock.json  
|-- terraform/  
|   |-- vpc.tf  
|   |-- security.tf  
|   |-- ec2.tf  
|   |-- sqs.tf  
|   |-- lambda.tf  
|   |-- iam.tf  
|   |-- provider.tf  
|   |-- variables.tf  
|   |-- terraform.tfvars  
|   |-- outputs.tf  
|-- images/  
|-- .gitignore  
|-- README.md
```

O .gitignore foi ajustado para impedir o envio do state do Terraform:

```
# Terraform  
.terraform/  
*.tfstate  
*.tfstate.*
```

IMPORTANTE

terraform.tfstate pode conter IDs, atributos e dados sensíveis. O state não deve ser tratado como um arquivo comum de código-fonte. Em equipes, usa-se backend remoto protegido e locking.

O histórico mostrou que os arquivos terraform.tfstate e terraform.tfstate.backup já estavam rastreados pelo Git; eles foram removidos do índice e o commit registrou a exclusão. Isso é diferente de apenas adicionar o padrão ao .gitignore: arquivos já rastreados continuam rastreados até serem removidos do índice.

Perguntas de revisão

1. Por que .terraform/ não deve ser versionada?
2. Por que adicionar *.tfstate ao .gitignore não basta se o arquivo já estava rastreado?
3. Qual pasta guarda as evidências visuais?
4. Qual arquivo documenta o projeto para a entrega?

5. Terraform e Infraestrutura como Código

O que fizemos

Organizamos os arquivos .tf, formatamos, validamos, planejamos e aplicamos a infraestrutura.

Por que fizemos

IaC torna a infraestrutura reproduzível, revisável e automatizável. Em vez de criar manualmente cada recurso no console, declaramos o estado desejado.

Como funciona

O provider AWS traduz recursos Terraform em chamadas à API da AWS. O state registra a correspondência entre recursos declarados e recursos existentes.

Passo a passo

- Entrar na pasta terraform.
- Executar terraform init na primeira configuração.
- Executar fmt e validate.
- Ler cuidadosamente o plan.
- Executar apply e confirmar com yes.

Comandos utilizados

```
cd /c/Projetos/aws-ecommerce-terraform/terraform
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

O que deveria acontecer

validate deveria retornar 'Success! The configuration is valid.'; plan mostraria adições/alterações/destruições; apply criaria ou modificaria recursos.

Como verificar

Executar terraform plan após o apply. Quando tudo está sincronizado, o esperado é 'No changes. Your infrastructure matches the configuration.'

Erros possíveis

Executar o comando na pasta errada gera 'No configuration files'. Texto colado com caracteres de bracketed paste pode produzir '[200~terraform: command not found'. Comandos concatenados, como 'terraform plan~cd', também falham.

O que aconteceu no nosso projeto

O projeto apresentou exatamente esses erros de terminal. A correção foi digitar os comandos limpos e entrar em /terraform antes de plan/apply. Também houve um plan com '3 to add, 1 to change, 0 to destroy' e, mais tarde, uma alteração de Security Group com '0 to add, 1 to change, 0 to destroy'.

O que preciso aprender

Memorize a sequência init/fmt/validate/plan/apply e, sobretudo, aprenda a ler o plan antes de confirmar. O plan é uma previsão; apply altera recursos reais.

Perguntas de revisão

1. Qual é a diferença entre terraform plan e terraform apply?
2. Por que o diretório atual importa ao executar Terraform?
3. Qual é o papel do terraform.tfstate?
4. O que significa '0 to add, 1 to change, 0 to destroy'?

6. Rede AWS: VPC, subnet, rotas e Security Group

O que fizemos

Provisionamos uma VPC, subnet pública, Internet Gateway, Route Table e Security Group.

Por que fizemos

A EC2 precisava existir dentro de uma rede e ser alcançável para os testes externos das APIs.

Como funciona

A VPC fornece o espaço de rede. A subnet divide esse espaço. A rota padrão envia tráfego externo ao Internet Gateway. O Security Group controla tráfego permitido para a instância.

Passo a passo

- Criar VPC e CIDR.
- Criar subnet pública na VPC.
- Criar e anexar Internet Gateway.
- Criar rota 0.0.0.0/0.
- Associar a Route Table à subnet.
- Criar Security Group e regras de entrada.

Comandos utilizados

```
# Regras observadas em security.tf
22  -> SSH
80   -> HTTP
3000 -> API de Produtos
3001 -> API de Pedidos
```

```
cidr_blocks = ["0.0.0.0/0"]
```

O que deveria acontecer

Após apply, a EC2 deveria aceitar conexões nas portas liberadas, desde que as aplicações estivessem ouvindo nelas.

Como verificar

Testar SSH e os endpoints. No console, conferir VPC, subnet, rotas, Internet Gateway e regras do Security Group.

Erros possíveis

Porta não liberada causa timeout externo. Aplicação ouvindo apenas em localhost também pode impedir acesso. Regra de SSH aberta ao mundo aumenta risco.

O que aconteceu no nosso projeto

A API de Produtos inicialmente não era acessível externamente e o navegador mostrou ERR_CONNECTION_TIMED_OUT. O Security Group foi atualizado para incluir a porta 3000; o Terraform mostrou uma alteração e o apply concluiu 'Resources: 0 added, 1 changed, 0 destroyed'. Depois o endpoint respondeu externamente.

O que preciso aprender

Entenda a diferença entre 'aplicação está rodando' e 'aplicação está acessível pela rede'. Ambas as condições precisam ser verdadeiras.

BOA PRÁTICA

Em ambiente real, restrinja a porta 22 ao seu IP em vez de 0.0.0.0/0. Exponha apenas as portas necessárias e considere um reverse proxy/Load Balancer com HTTPS.

Perguntas de revisão

1. Por que liberar a porta 3000 resolveu apenas uma parte do problema de acesso?
2. O que significa 0.0.0.0/0?
3. Qual é a função do Internet Gateway?
4. Por que SSH aberto para toda a internet não é recomendado?

7. EC2, Linux, SSH e execução das APIs

O que fizemos

Acessamos a instância EC2 por SSH e executamos as aplicações Node.js no Linux.

Por que fizemos

A EC2 era o servidor de aplicação. Precisávamos administrar arquivos, instalar dependências e iniciar as APIs.

Como funciona

SSH cria uma sessão remota criptografada. A chave privada .pem prova posse da credencial. Dentro da EC2, os processos Node.js escutam portas TCP.

Passo a passo

- Usar a chave .pem correta.
- Conectar ao IPv4 público.
- Entrar na pasta da API.
- Instalar dependências quando necessário.
- Executar node index.js.

Comandos utilizados

```
ssh -i "caminho/mercece-key.pem" ec2-user@IP_PUBLICO
cd ~/products-api
npm install
node index.js
```

O que deveria acontecer

O terminal da EC2 deveria indicar a API rodando, por exemplo 'API rodando na porta 3000'.

Como verificar

Na própria EC2, usar curl localhost. Externamente, usar o IP público e a porta liberada.

Erros possíveis

Chave errada/permisões da chave, IP alterado, porta 22 fechada ou processo encerrado podem impedir acesso. Versão antiga do Node pode causar incompatibilidade com pacotes.

O que aconteceu no nosso projeto

O histórico mostrou Node v16.20.2 e avisos npm EBADENGINE porque alguns pacotes exigiam Node >=18. Esse ponto se tornou relevante no erro do SDK/SQS; o ambiente precisou ser ajustado para uma versão compatível.

O que preciso aprender

Separe diagnóstico de rede, sistema operacional e aplicação. Teste primeiro localmente na EC2, depois externamente.

Perguntas de revisão

1. O que a opção -i do ssh indica?
2. Por que testar localhost na EC2 antes do IP público?
3. O que um aviso EBADENGINE indica?
4. O que acontece com uma API iniciada em primeiro plano se a sessão for encerrada?

8. API de Produtos

A API REST de Produtos foi executada na EC2 na porta 3000. O endpoint principal confirmado foi GET /produtos.

```
curl http://IP_PUBLICO_EC2:3000/produtos
```

Resposta observada:

```
[
  {"id":1,"nome":"Notebook","preco":3500},
  {"id":2,"nome":"Mouse","preco":80},
  {"id":3,"nome":"Teclado","preco":150}
]
```

O navegador também exibiu esse JSON, servindo como evidência de que o processo Node.js estava ativo, a porta estava liberada e havia conectividade externa.

O que preciso aprender

Uma resposta HTTP bem-sucedida depende de várias camadas: aplicação ouvindo, sistema operacional funcional, Security Group permitindo tráfego, rota e IP público corretos.

Perguntas de revisão

1. Qual método HTTP foi usado para listar produtos?
2. Qual porta foi utilizada pela API de Produtos?
3. O que a resposta JSON comprovou além do funcionamento do código?
4. Qual diferença existe entre testar localhost e IP público?

9. API de Pedidos e Amazon SQS

O que fizemos

Executamos a API de Pedidos na porta 3001 e enviamos um POST com `produtoId` e `quantidade`. A API enviou o pedido para a fila SQS.

Por que fizemos

O requisito central era demonstrar mensageria assíncrona. A API não precisava executar todo o processamento final durante a requisição.

Como funciona

A aplicação cria um objeto de pedido e usa o AWS SDK para enviar uma mensagem. O SQS armazena a mensagem até um consumidor processá-la.

Passo a passo

- Garantir que a API esteja rodando.
- Garantir permissão IAM para `SendMessage`.
- Enviar POST /pedidos.
- Observar a resposta.
- Conferir métricas da fila.

Comandos utilizados

```
curl -X POST http://IP_PUBLICO_EC2:3001/pedidos \
-H "Content-Type: application/json" \
-d '{"produtoId":1,"quantidade":2}'
```

O que deveria acontecer

A resposta final observada foi: `{"message":"Pedido criado e enviado para a fila SQS!", "pedido":{"id":4, "produtoId":1, "quantidade":2, "status":"recebido"}}`

Como verificar

No SQS, as métricas `NumberOfMessagesSent/Received/Deleted` e as métricas aproximadas indicaram atividade da fila.

Erros possíveis

Falhas de credencial/permissão, URL/região incorreta, SDK incompatível ou consumidor indisponível podem impedir o fluxo.

O que aconteceu no nosso projeto

No início, a API respondeu 'Erro ao enviar pedido para o SQS'. O log mostrou `TypeError: getRandomValues is not a function` dentro de dependências do AWS SDK. O ambiente estava com Node 16 e pacotes que exigiam Node ≥ 18 . Após a correção do runtime/dependências, o mesmo curl retornou sucesso e o pedido id 4 foi enviado.

O que preciso aprender

Leia o stack trace de baixo para cima até encontrar seu arquivo e a biblioteca envolvida. Versões de runtime são parte da compatibilidade da aplicação.

Perguntas de revisão

1. Por que SQS desacopla produtor e consumidor?
2. Qual método HTTP criou o pedido?
3. O que significam as métricas Sent, Received e Deleted?
4. Qual foi a causa técnica associada ao erro `getRandomValues`?

10. IAM: permissões entre serviços

IAM foi necessário porque estar na mesma conta AWS não concede automaticamente acesso entre recursos. A EC2 precisava autorização para enviar mensagens; a Lambda precisava consumir a fila e escrever logs.

Princípio do menor privilégio

Uma policy deve permitir apenas as ações e recursos necessários. Para a EC2, por exemplo, a necessidade principal era enviar ao SQS. Para a Lambda, consumir mensagens e usar CloudWatch Logs.

SEGURANÇA

Evite access key e secret key fixas em código, README, .env versionado ou prints públicos. Em EC2, prefira IAM Role/Instance Profile; em Lambda, use Execution Role.

Como verificar

Quando a API envia ao SQS sem erro de AccessDenied e a Lambda consome e registra logs, há forte evidência de que as permissões essenciais estão corretas. No console IAM, também é possível inspecionar roles e policies.

Perguntas de revisão

1. Por que a EC2 precisa de uma role para acessar SQS?
2. O que é menor privilégio?
3. Por que não colocar secret key no código?
4. Quais permissões gerais a Lambda precisava no fluxo?

11. AWS Lambda e CloudWatch Logs

O que fizemos

Criamos uma função Lambda acionada pela fila SQS. A função leu a mensagem e registrou o pedido no CloudWatch.

Por que fizemos

Essa etapa fechava o fluxo assíncrono exigido pela atividade.

Como funciona

O mapeamento de evento faz a Lambda consultar/receber mensagens do SQS. Cada invocação recebe registros; o código interpreta o body e escreve logs.

Passo a passo

- Empacotar o código da Lambda.
- Provisionar função e role.
- Configurar trigger/event source mapping SQS.
- Enviar pedido.
- Abrir o log group da função.

Comandos utilizados

```
# Evidência conceitual do log
INFO Evento recebido do SQS:
INFO Pedido recebido: {"id":4,"produtoId":1,"quantidade":2,"status":"recebido"}
```

O que deveria acontecer

Uma nova invocação deveria aparecer no CloudWatch com START, mensagens INFO, END e REPORT.

Como verificar

No log stream, conferir timestamp, RequestId, mensagem recebida e ausência de erro.

Erros possíveis

Permissões insuficientes, trigger ausente/desabilitado, código inválido ou mensagens em retry/DLQ podem impedir o processamento.

O que aconteceu no nosso projeto

O CloudWatch mostrou runtime nodejs:20.v101 e o pedido id 4 com produtoId 1 e quantidade 2. A execução terminou normalmente e o REPORT mostrou duração e memória utilizada. Isso confirmou o fluxo SQS -> Lambda -> CloudWatch.

O que preciso aprender

Aprenda a usar logs como evidência operacional: não basta saber que a função existe; é preciso comprovar que foi invocada e processou a mensagem esperada.

Perguntas de revisão

1. Qual evidência confirmou que a Lambda recebeu o pedido?
2. O que representam START, END e REPORT?
3. Por que CloudWatch é parte da observabilidade?
4. Qual era o papel do trigger SQS?

12. Testes ponta a ponta e evidências

As evidências reunidas no README cobriram as quatro exigências principais da entrega.

Evidência	O que comprovou
EC2/API de Produtos	A aplicação Node.js estava acessível externamente na porta 3000 e retornava JSON.
Monitoramento SQS	Mensagens foram enviadas, recebidas e consumidas pela fila.
CloudWatch Lambda	A Lambda foi acionada pelo SQS e registrou o pedido processado.
terraform apply	A infraestrutura foi alterada com sucesso pelo Terraform, sem destruição naquela atualização.

Teste ponta a ponta

- 1) GET `http://IP:3000/produtos`
- 2) POST `http://IP:3001/pedidos`
- 3) Conferir SQS
- 4) Conferir Lambda/CloudWatch
- 5) Conferir `terraform plan`

Perguntas de revisão

1. Qual evidência prova a camada de aplicação?
2. Qual evidência prova a mensageria?
3. Qual evidência prova o processamento serverless?
4. Por que uma captura do `terraform apply` é relevante para uma atividade de IaC?

13. Git, GitHub, README e entrega

O que fizemos

Atualizamos .gitignore e README, adicionamos código Lambda, arquivos Terraform e imagens de evidência, criamos commit e enviamos para origin/main.

Por que fizemos

O GitHub era a entrega técnica: permitia ao professor inspecionar código, documentação e evidências.

Como funciona

Git mantém snapshots locais; git add prepara alterações, commit grava um snapshot e push envia commits ao repositório remoto.

Passo a passo

- Verificar git status.
- Adicionar alterações.
- Criar commit descritivo.
- Enviar para origin main.
- Abrir GitHub e conferir arquivos/imagens/README.

Comandos utilizados

```
git status
git add .
git commit -m "feat: finaliza infraestrutura AWS com SQS Lambda e evidencias"
git push origin main
```

O que deveria acontecer

O push deveria mostrar main -> main e os arquivos deveriam aparecer no GitHub.

Como verificar

Abrir o repositório remoto, navegar pelas pastas e confirmar que README e imagens renderizam.

Erros possíveis

Branch digitada incorretamente causa 'src refspec ... does not match any'. Arquivos sensíveis podem ser enviados se o .gitignore estiver incompleto.

O que aconteceu no nosso projeto

Foi digitado acidentalmente 'git push origin maingit push origin main', gerando erros de refspec. Em seguida, 'git push origin main' funcionou e enviou o commit 3a74ae0. O commit registrou 14 arquivos alterados e removeu os tfstate do versionamento.

O que preciso aprender

Antes de push, leia git status. Comandos Git são simples, mas erros de digitação podem mudar completamente a interpretação.

Perguntas de revisão

1. Qual é a função de git add, commit e push?
2. Por que o git status deve ser consultado antes do commit?
3. O que causou o erro src refspec no projeto?
4. Por que tfstate não deve permanecer no GitHub?

14. Projeto do zero ao deploy

Este capítulo reorganiza a experiência em um tutorial limpo. Ajuste AMI, região, nomes e IDs conforme seu ambiente.

PASSO 1 - Preparar o repositório

- Criar as pastas products-api, orders-api, lambda, terraform e images.
- Adicionar .gitignore desde o início.
- Nunca versionar .pem, .env, .terraform ou tfstate.

PASSO 2 - Criar as APIs Node.js

- API Produtos: GET /produtos na porta 3000.
- API Pedidos: POST /pedidos na porta 3001.
- Na API de Pedidos, configurar cliente SQS e SendMessageCommand usando AWS SDK v3.

PASSO 3 - Declarar provider e variáveis

- Definir região AWS.
- Definir project_name e demais variáveis.
- Evitar segredos em terraform.tfvars versionado.

PASSO 4 - Declarar rede

- VPC.
- Subnet pública.
- Internet Gateway.
- Route Table com rota padrão.
- Associação da rota.

PASSO 5 - Declarar Security Group

- 22 para SSH, preferencialmente restrito ao seu IP.
- 3000 e 3001 para o laboratório.
- Egress necessário.

PASSO 6 - Declarar SQS

- Criar fila orders-queue.
- Expor URL/ARN como outputs quando útil.

PASSO 7 - Declarar IAM

- Role/Instance Profile da EC2 com permissão mínima para SQS.
- Execution Role da Lambda com SQS e CloudWatch Logs.

PASSO 8 - Declarar EC2

- AMI compatível.
- Subnet e Security Group.
- Chave SSH.
- Associar Instance Profile.

PASSO 9 - Declarar Lambda e trigger

- Empacotar lambda/index.js em ZIP.
- Criar aws_lambda_function.
- Criar event source mapping da fila.

PASSO 10 - Validar e aplicar

```
cd terraform
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

PASSO 11 - Preparar a EC2

```
ssh -i "chave.pem" ec2-user@IP_PUBLICO
cd ~/products-api
npm install
node index.js
```

PASSO 12 - Testar Produtos

```
curl http://IP_PUBLICO:3000/produtos
```

PASSO 13 - Testar Pedidos

```
curl -X POST http://IP_PUBLICO:3001/pedidos \
-H "Content-Type: application/json" \
-d '{"produtoId":1,"quantidade":2}'
```

PASSO 14 - Verificar SQS e Lambda

- Conferir métricas da fila.
- Abrir CloudWatch Logs e localizar o pedido.

PASSO 15 - Documentar

- Salvar prints em images/.
- Atualizar README com arquitetura, comandos, testes e evidências.
- Commit e push.

PASSO 16 - Encerrar

```
cd terraform
terraform destroy
# confirmar: yes
```

- Conferir console e Billing/Cost Explorer.
- Auditar outras regiões.

Perguntas de revisão

1. Por que IAM deve ser criado antes de depender dele na EC2/Lambda?
2. Em que momento o teste local deve preceder o teste externo?
3. Qual é a ordem lógica do teste ponta a ponta?
4. Por que o destroy vem somente após a coleta das evidências?

15. Erros reais e troubleshooting

Erros que realmente aconteceram

Problema	Possível causa	Como identificamos	Solução	Como evitar
[200~terraform: command not found	Caracteres de bracketed paste no terminal	Bash mostrou o prefixo [200~	Redigitar comando limpo	Colar com cuidado; conferir linha antes de Enter
terraform plan~cd	Dois comandos colados/concatenados	Terraform disse que plan~cd não existe	Executar cd e terraform plan separadamente	Uma instrução por linha
No configuration files	Terraform executado na raiz do repo	plan informou ausência de .tf	cd terraform	Sempre conferir pwd/caminho
ERR_CONNECTION_TIMED_OUT :3000	Porta 3000 não permitida no SG	API local existia, acesso externo falhava	Adicionar regra 3000 e terraform apply	Planejar portas antes do deploy
EBADENGINE / Node 16	Dependências exigiam Node >=18	npm exibiu warnings	Usar runtime compatível	Ver engines e versão do Node
getRandomValues is not a function	Incompatibilidade runtime/dependências AWS SDK	Stack trace em @smithy/core/@aws-sdk	Atualizar runtime/dependências	Padronizar Node LTS suportado
Erro ao enviar pedido para o SQS	Falha no envio decorrente do erro acima	API retornou JSON de erro	Corrigir runtime e retestar	Logar erro completo no servidor
src refspec ... does not match any	Comando push digitado incorretamente	Git mostrou refspec maingit/push	git push origin main	Conferir comando/branch
tfstate aparecendo no git status	State já havia sido rastreado	git status mostrou arquivos modificados	Ignorar e remover do índice; commit da remoção	Criar .gitignore antes do primeiro commit

Troubleshooting complementar

EXPLICAÇÃO COMPLEMENTAR Os itens abaixo são problemas comuns de projetos semelhantes; não são apresentados como fatos ocorridos no projeto.

- AccessDenied no SQS: revisar role, policy, ARN e Instance Profile.
- Lambda não dispara: revisar event source mapping, permissões e estado Enabled.
- SSH timeout: revisar IP público, porta 22, rota, Internet Gateway e NACL.
- Connection refused: normalmente há conectividade até o host, mas nenhum processo escuta naquela porta.
- Terraform drift: mudanças manuais no console podem aparecer no próximo plan.
- Mensagem presa no SQS: verificar consumidor, visibility timeout, retries e possíveis erros da Lambda.

Perguntas de revisão

1. Qual diferença diagnóstica existe entre timeout e connection refused?
2. Por que o stack trace foi importante no erro do SQS?
3. Como evitar executar Terraform na pasta errada?
4. O que é drift em Terraform?

16. Segurança e boas práticas

IAM e credenciais

- Use roles temporárias em vez de access keys fixas.
- Aplique menor privilégio.
- Nunca publique chave .pem, secret access key ou tokens.

Security Groups e SSH

- No laboratório, regras amplas facilitaram testes.
- Em produção, restrinja SSH ao IP administrativo.
- Evite expor diretamente portas internas quando um proxy/Load Balancer puder centralizar acesso.

Git e arquivos sensíveis

```
node_modules/  
.env  
.env.*  
.terraform/  
*.tfstate  
*.tfstate.*  
*.pem
```

Terraform state

O state merece proteção especial. Em times, use backend remoto com controle de acesso, criptografia e locking. Não o envie para um repositório público.

Conexão com AWS/Cloud

Segurança na nuvem é responsabilidade compartilhada: a AWS protege a infraestrutura física e serviços gerenciados; o cliente ainda precisa configurar IAM, rede, dados, sistemas e aplicações corretamente.

Perguntas de revisão

1. O que nunca deve ser enviado ao GitHub?
2. Por que role é melhor que chave fixa na EC2?
3. Qual regra de Security Group do laboratório merece maior restrição em produção?
4. Por que state é considerado sensível?

17. Docker e Dockerfile: o que não usamos

REALIZADO NO PROJETO

Não foi confirmado uso de Docker, Dockerfile, imagem Docker ou container. As APIs foram iniciadas diretamente com Node.js na EC2. Portanto, Docker não fazia parte do fluxo final exigido que reconstruímos.

O que é Docker?

Docker é uma plataforma de empacotamento e execução de aplicações em containers. Um container isola processo, filesystem e dependências, compartilhando o kernel do host.

Imagem x container

Imagem é um artefato imutável usado como molde. Container é uma instância em execução dessa imagem. Uma mesma imagem pode originar vários containers.

Dockerfile

Dockerfile é um arquivo de instruções para construir uma imagem. Como não há Dockerfile confirmado neste projeto, não existe uma lista real de instruções do nosso trabalho para explicar linha a linha.

Exemplo complementar - NÃO pertence ao projeto executado

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

Leitura do exemplo

- FROM escolhe a imagem-base.
- WORKDIR define o diretório de trabalho.
- COPY copia arquivos.
- RUN executa comando durante o build.
- EXPOSE documenta a porta da aplicação.
- CMD define o processo padrão do container.

Comandos Docker de referência

```
docker build -t products-api .
docker run -d --name products-api -p 3000:3000 products-api
docker ps
docker logs products-api
docker stop products-api
docker rm products-api
docker rmi products-api
```

IMPORTANTE

Esses comandos são material complementar para estudo. Não os trate como evidência de que Docker foi utilizado na atividade.

Perguntas de revisão

1. Docker foi confirmado no projeto executado?
2. Qual é a diferença entre imagem e container?
3. O que -p 3000:3000 faz em docker run?
4. Qual instrução define o processo padrão no exemplo de Dockerfile?

18. Custos AWS e encerramento seguro

Ao final, executamos terraform destroy. O plano indicou 16 recursos para destruir e o terminal confirmou: 'Destroy complete! Resources: 16 destroyed.' A fila SQS levou mais tempo que outros recursos, mas também concluiu a destruição.

O que foi verificado depois

- VPC: a tela mostrou apenas uma VPC padrão com CIDR 172.31.0.0/16, sem o nome do projeto.
- CloudWatch: permaneceu um log group da Lambda. Log groups podem sobreviver à exclusão da função se não forem gerenciados explicitamente pelo Terraform.
- Billing: a tela mostrava custo estimado do mês e a conta em plano gratuito/créditos; depois, Bills mostrava total estimado USD 0.00 para o período exibido.
- Cost Explorer: agrupamos por região e observamos regiões como US East (N. Virginia), US East (Ohio) e 'No region', todas com USD 0.00 no intervalo mostrado.

Parar, encerrar, excluir, desassociar e liberar

- Parar: mantém o recurso existente, mas interrompe execução. Ex.: EC2 parada pode deixar armazenamento cobrando.
- Encerrar (terminate): remove a instância EC2; volumes configurados para persistir podem sobreviver.
- Excluir: remove um recurso, como fila, função ou log group.
- Desassociar: rompe vínculo sem necessariamente apagar os recursos, como uma associação de endereço/rede.
- Liberar: devolve um recurso alocado, como Elastic IP, quando aplicável.

Recursos do projeto com potencial de custo

- EC2 e volumes EBS.
- IPv4 público, conforme modelo de cobrança vigente da AWS.
- SQS por requisições além de franquias/créditos.
- Lambda por invocações e duração além de franquias/créditos.
- CloudWatch Logs por ingestão/armazenamento.
- Transferência de dados.

Checklist de encerramento seguro da AWS

```
[x] Terraform destroy concluído: 16 recursos destruídos
[x] EC2 do projeto removida
[x] SQS do projeto removido
[x] Lambda do projeto removida
[x] VPC do projeto removida
[x] Security Group/rotas/subnet/IGW gerenciados removidos
[!] CloudWatch Log Group: verificar/remover se não precisar manter evidência
```

Verificações preventivas:

```
[ ] Volumes EBS órfãos
[ ] Snapshots
[ ] Elastic IP
[ ] Load Balancers
[ ] NAT Gateway
[ ] RDS
[ ] S3 buckets
[ ] Recursos em outras regiões
[ ] Billing / Bills / Cost Explorer
[ ] Budgets / alertas de custo
```

ATENÇÃO

A ausência de recursos no Terraform não garante que nunca existam recursos criados manualmente fora do state. Por isso a auditoria final inclui console, regiões e Billing.

Auditoria final recomendada

- No Cost Explorer, agrupar por Region e por Service.
- Verificar EC2 > Instances e Volumes.
- Verificar Elastic IPs.
- Verificar Load Balancers/NAT Gateways apenas como prevenção.
- Verificar Lambda, SQS e CloudWatch Logs.
- Repetir em regiões que apresentem uso/custo.
- Criar Budget de baixo valor para alerta futuro.

Perguntas de revisão

1. Por que parar EC2 não é igual a excluir todos os custos relacionados?
2. Qual recurso permaneceu visível após o destroy?
3. Por que verificar Cost Explorer por região?
4. O que significa o destroy ter confirmado 16 recursos destruídos?

19. Glossário

AWS - Amazon Web Services, plataforma de computação em nuvem.

AMI - Imagem usada como base para iniciar uma instância EC2.

API REST - Interface HTTP organizada em recursos/endpoints e métodos como GET e POST.

ARN - Identificador único de um recurso AWS.

CloudWatch Logs - Serviço de coleta e armazenamento de logs.

CIDR - Notação que representa um bloco de endereços IP.

EC2 - Serviço de máquinas virtuais da AWS.

EBS - Armazenamento em bloco normalmente associado à EC2.

Endpoint - Endereço de acesso a uma operação de API.

Event source mapping - Configuração que conecta uma fonte de eventos, como SQS, a uma Lambda.

Git - Sistema distribuído de controle de versão.

GitHub - Hospedagem remota de repositórios Git.

IAM - Serviço de identidades, roles e permissões da AWS.

IaC - Infraestrutura como Código.

Internet Gateway - Gateway que permite comunicação entre VPC e internet quando corretamente roteado.

IPv4 público - Endereço alcançável pela internet.

IPv4 privado - Endereço usado dentro da rede privada/VPC.

Lambda - Serviço serverless para execução de funções.

Node.js - Runtime JavaScript usado nas APIs e Lambda.

npm - Gerenciador de pacotes do ecossistema Node.js.

Policy - Documento de permissões IAM.

Porta - Número lógico usado para direcionar conexões a serviços/processos.

Provider Terraform - Plugin que permite ao Terraform gerenciar uma plataforma, como AWS.

Route Table - Tabela de roteamento da VPC.

Security Group - Firewall stateful associado a recursos AWS.

SSH - Protocolo seguro para acesso remoto ao Linux.

SQS - Serviço de filas gerenciado da AWS.

Subnet - Sub-rede dentro de uma VPC.

Terraform state - Arquivo/base que registra o estado conhecido da infraestrutura.

VPC - Rede virtual privada dentro da AWS.

Visibility Timeout - Período em que uma mensagem SQS recebida fica temporariamente invisível para outros consumidores.

Docker - Plataforma de containers; não confirmada na execução deste projeto.

Dockerfile - Receita para construir imagem Docker; não confirmada na execução deste projeto.

20. Folha de consulta rápida de comandos

Linux / navegação

```
cd caminho
ls
ls -la
cat arquivo
node index.js
npm install
```

Terraform

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform destroy
```

Git

```
git status
git add .
git commit -m "mensagem"
git push origin main
```

SSH e aplicação

```
ssh -i "chave.pem" ec2-user@IP_PUBLICO
```

```
curl http://IP_PUBLICO:3000/produtos
```

```
curl -X POST http://IP_PUBLICO:3001/pedidos \
-H "Content-Type: application/json" \
-d '{"produtoId":1,"quantidade":2}'
```

Docker - somente referência complementar

```
docker build -t nome .
docker run -d -p 3000:3000 nome
docker ps
docker logs container
docker stop container
docker rm container
docker rmi imagem
```

21. Perguntas de revisão e gabarito comentado

As perguntas ao final de cada capítulo devem ser respondidas antes de consultar esta seção. Abaixo estão os pontos essenciais que uma boa resposta deveria conter.

Capítulos 1 a 6

- Terraform: descreve/provisiona infraestrutura; plan prevê, apply executa; state relaciona código e recursos.
- APIs: Produtos e Pedidos; SQS desacopla envio e processamento.
- EC2 hospeda APIs; Lambda processa eventos; CloudWatch registra evidências.
- VPC/subnet/rotas/IGW formam a conectividade; Security Group controla portas.
- 0.0.0.0/0 significa qualquer IPv4; SSH deveria ser restrito.
- .gitignore deve excluir state e arquivos sensíveis antes do primeiro commit.

Capítulos 7 a 13

- SSH usa chave privada; teste localhost separa problema da aplicação de problema de rede.
- GET /produtos na porta 3000 comprovou aplicação e conectividade.
- POST /pedidos na porta 3001 enviou ao SQS; métricas Sent/Received/Deleted mostraram atividade.
- IAM Role evita credenciais fixas e deve seguir menor privilégio.
- CloudWatch mostrou START/INFO/END/REPORT da Lambda.
- Git add prepara, commit registra, push publica; o erro refsPEC veio de comando digitado incorretamente.

Capítulos 14 a 18

- O fluxo limpo é preparar código -> declarar IaC -> apply -> configurar/executar apps -> testar -> documentar -> destroy.
- Timeout sugere ausência de resposta de rede; connection refused costuma indicar host alcançado sem serviço escutando.
- Docker não foi confirmado no projeto; imagem é molde e container é instância em execução.
- Parar não equivale a remover. Após destroy, audite volumes, IPs, logs, outras regiões e Billing.
- O log group remanescente deve ser avaliado/removido se não houver necessidade de retenção.

22. Simulado final - 30 questões

1. (Múltipla escolha) Qual serviço hospedou as APIs? A) S3 B) EC2 C) Lambda D) RDS
2. (V/F) O Terraform plan modifica imediatamente a infraestrutura.
3. Qual componente recebeu as mensagens da API de Pedidos? A) SNS B) SQS C) EBS D) Route 53
4. (V/F) A Lambda foi acionada pelo SQS.
5. Qual porta foi usada pela API de Produtos? A) 22 B) 80 C) 3000 D) 3001
6. Qual porta foi usada pela API de Pedidos? A) 3001 B) 443 C) 53 D) 8080
7. O que controla tráfego de entrada da EC2? A) IAM B) Security Group C) CloudWatch D) Git
8. (V/F) 0.0.0.0/0 em uma regra IPv4 representa qualquer origem IPv4.
9. Qual comando valida sintaxe/configuração Terraform? A) terraform test B) terraform validate C) terraform check D) terraform sync
10. O erro 'No configuration files' ocorreu porque: A) AWS caiu B) comando foi executado fora da pasta com .tf C) SQS estava vazio D) Git estava sujo
11. (V/F) terraform.tfstate é recomendado para repositório público.
12. Qual método HTTP foi usado para criar pedidos? A) GET B) POST C) DELETE D) PATCH
13. Qual biblioteca permitiu à API chamar SQS? A) React B) AWS SDK for JavaScript C) Nginx D) Git
14. O warning EBADENGINE indicava: A) disco cheio B) versão de Node incompatível C) porta fechada D) IAM negado
15. O erro getRandomValues esteve associado principalmente a: A) compatibilidade runtime/dependências B) VPC sem CIDR C) README D) GitHub
16. (V/F) IAM Role é preferível a gravar access key no código.
17. Qual serviço armazenou os logs da Lambda? A) CloudWatch Logs B) S3 obrigatoriamente C) EC2 D) IAM
18. O que 'NumberOfMessagesDeleted' ajuda a indicar? A) mensagens removidas após processamento B) instâncias terminadas C) arquivos Git apagados D) VPCs excluídas
19. (V/F) Dockerfile foi confirmado como requisito/uso da execução reconstruída.
20. Imagem Docker é: A) container em execução B) molde para criar containers C) fila D) policy IAM
21. Qual comando publicou os commits? A) git apply B) git push origin main C) git plan D) git send
22. O erro src refspec aconteceu por: A) comando/branch digitado incorretamente B) SQS sem mensagens C) porta 3000 D) CloudWatch
23. (V/F) Parar uma EC2 garante que nenhum recurso associado possa gerar custo.
24. Qual comando remove os recursos gerenciados pelo Terraform? A) terraform clean B) terraform destroy C) terraform erase D) terraform stop
25. Quantos recursos o destroy confirmou no projeto? A) 4 B) 8 C) 16 D) 30
26. Qual recurso permaneceu visível após o destroy? A) API rodando B) log group CloudWatch C) EC2 do projeto D) SQS do projeto
27. Para investigar gastos por localização, Cost Explorer pode agrupar por: A) Region B) README C) Git branch D) porta
28. Situação: curl localhost funciona, mas IP público dá timeout. Primeiro revisar: A) Security Group/rede B) package-lock apenas C) Git commit D) nome do README
29. Situação: Lambda existe, mas não processa SQS. Uma verificação essencial é: A) event source mapping/trigger B) porta 3000 C) git status D) navegador
30. Qual sequência representa o fluxo final? A) Usuário->EC2->SQS->Lambda->CloudWatch B) Usuário->S3->RDS->EC2 C) Git->EC2->Docker->S3 D) EC2->Usuário->IAM->SQS

23. Gabarito comentado do simulado

Q	Resp.	Comentário
1	B	EC2 foi o servidor das duas APIs.
2	F	plan calcula e exibe ações; apply é que altera recursos.
3	B	A API de Pedidos enviou mensagens para Amazon SQS.
4	V	O trigger/event source mapping conectou SQS à Lambda.
5	C	Produtos foi exposta na porta 3000.
6	A	Pedidos foi exposta na porta 3001.
7	B	Security Group funciona como firewall stateful do recurso.
8	V	A origem é qualquer endereço IPv4.
9	B	terraform validate verifica a configuração.
10	B	O comando foi executado na raiz sem arquivos .tf.
11	F	State pode conter dados sensíveis e não deve ir para repositório público.
12	B	POST /pedidos criou o pedido.
13	B	O AWS SDK for JavaScript realizou a chamada ao SQS.
14	B	Pacotes exigiam Node >=18 e o ambiente mostrava Node 16.
15	A	O stack trace apontou incompatibilidade em dependências do SDK/Smithy.
16	V	Roles entregam credenciais temporárias e evitam segredos fixos.
17	A	CloudWatch Logs armazenou os eventos da função.
18	A	Ajuda a observar mensagens removidas da fila após consumo bem-sucedido.
19	F	Docker não foi confirmado no histórico do projeto.
20	B	Imagem é o artefato-base; container é uma instância dessa imagem.
21	B	git push origin main enviou o commit ao remoto.
22	A	Houve concatenação/digitação incorreta do comando de push.
23	F	Volumes, IPs e outros recursos podem continuar existindo/cobrando.
24	B	terraform destroy planeja e remove recursos gerenciados.
25	C	O terminal confirmou 16 recursos destruídos.
26	B	O log group da Lambda ainda aparecia no CloudWatch.
27	A	Agrupar por Region ajuda a localizar uso/custo por região.
28	A	Se local funciona e externo dá timeout, rede/SG é forte suspeita.
29	A	Sem trigger habilitado, a função não receberá eventos da fila.
30	A	Esse foi o fluxo final exigido e comprovado.

24. Desafio final: refaça o projeto sozinho

Cenário

Você recebeu a tarefa de criar uma pequena arquitetura de pedidos em nuvem. Uma API de catálogo e uma API de pedidos devem rodar em EC2. A criação de pedido deve publicar uma mensagem em uma fila. Uma função serverless deve processar a mensagem e registrar logs.

Requisitos

- Infraestrutura declarada em Terraform.
- VPC e subnet com conectividade adequada.
- EC2 com duas APIs Node.js.
- Security Group com portas estritamente necessárias.
- SQS para pedidos.
- IAM sem credenciais fixas no código.
- Lambda acionada pelo SQS.
- CloudWatch Logs.
- README e evidências.
- Encerramento seguro.

Critérios de conclusão

```
[ ] terraform validate sem erros
[ ] terraform plan compreendido
[ ] APIs acessíveis
[ ] GET /produtos retorna JSON
[ ] POST /pedidos retorna sucesso
[ ] SQS registra atividade
[ ] Lambda processa a mensagem
[ ] CloudWatch mostra o pedido
[ ] README documentado
[ ] GitHub atualizado sem segredos/state
[ ] terraform destroy concluído
[ ] auditoria de custos/regiões concluída
```

Solução sugerida - consulte somente depois de tentar

Estruture o Terraform por responsabilidade (provider/variáveis, rede, segurança, EC2, SQS, IAM e Lambda). Use role da EC2 para SendMessage e execution role da Lambda para SQS/Logs. Teste primeiro as APIs localmente na EC2, depois externamente. Em seguida teste o POST, observe a fila e confirme o log da Lambda. Documente as quatro evidências e só então execute destroy e a auditoria de custos.

Checklist de domínio final

- Consigo desenhar o fluxo sem consultar a apostila.
- Consigo explicar por que cada recurso existe.
- Consigo interpretar plan/apply/destroy.
- Consigo diferenciar falha de aplicação, rede e permissão.
- Consigo explicar por que SQS e Lambda formam um fluxo assíncrono.
- Consigo proteger state e credenciais.
- Consigo auditar recursos e custos após o laboratório.

RESULTADO ESPERADO

Se você conseguir refazer o desafio e explicar as decisões em voz alta, o projeto deixou de ser apenas uma atividade executada e passou a ser conhecimento reutilizável.